Projekt Zespołowy

Raport 2 – Postępy Prac

Inteligentny System Automatycznego Podlewania
i Doświetlania Roślin
z Analizą Warunków Środowiskowych

Wykonawcy projektu:	
Student A (programowanie):	Konrad Husiatyński
Student B (hardware):	Sebastian Szymczyk
Data:	30.04.2026

1. Wprowadzenie

Niniejszy raport stanowi dokumentację postępów prac nad projektem zespołowym realizowanym w ramach przedmiotu Projekt Zespołowy. Raport przedstawia stopień zaawansowania prac w stosunku do założeń przedstawionych w Raporcie 1. Całość też jest aktualnie umieszczana w repozytorium na github: https://github.com/khusiatynski/smart-plant-system .

Oraz zgodnie z zaleceniami zrobiliśmy stronę na github:

https://khusiatynski.github.io/smart-plant-system/

Przypomnienie założeń projektu:

- Opracowanie systemu monitorowania i sterowania warunkami wzrostu roślin
- Pomiar parametrów środowiskowych (wilgotność gleby, temperatura, wilgotność powietrza, ciśnienie, natężenie światła, poziom wody)
- Automatyczne sterowanie pompą wodną i oświetleniem LED
- Logowanie danych pomiarowych do pliku CSV
- Aplikacja operatorska z interfejsem graficznym

2. Zrealizowane Funkcjonalności

2.1. Warstwa sprzętowa

Zakupiono i skompletowano wszystkie elementy układu:

Lp.	Komponent	Status
1	Arduino Nano V3 (CH340, USB-C)	✓ Zakupione
2	BME280 (temp., wilg., ciśn.)	✓ Zakupione
3	Czujnik wilgotności gleby M335	✓ Zakupione
4	Czujnik poziomu wody	✓ Zakupione
5	Czujnik światła LM393	✓ Zakupione
6	Moduł PWM 2x MOSFET 400W	✓ Zakupione
7	Pompa DC JT-DC3W (3-5V)	✓ Zakupione
8	LED 3W Epistar Full Spectrum	✓ Zakupione
9	Czytnik kart SD	✓ Zakupione

2.2. Warstwa programowa

Opracowano pełne oprogramowanie systemu:

- Kod Arduino (sketch_gui_sync.ino) – sterowanie czujnikami i elementami wykonawczymi z synchronizacją UART
- Aplikacja GUI w Python (tkinter) – panel operatorski z zegarami BME, wykresami trendów, sterowaniem ręcznym
- Symulacja Wokwi – działającjhka kopia układu do testów online
- System logowania – zapis danych do CSV z sygnaturami czasowymi

2.3. Dokumentacja techniczna

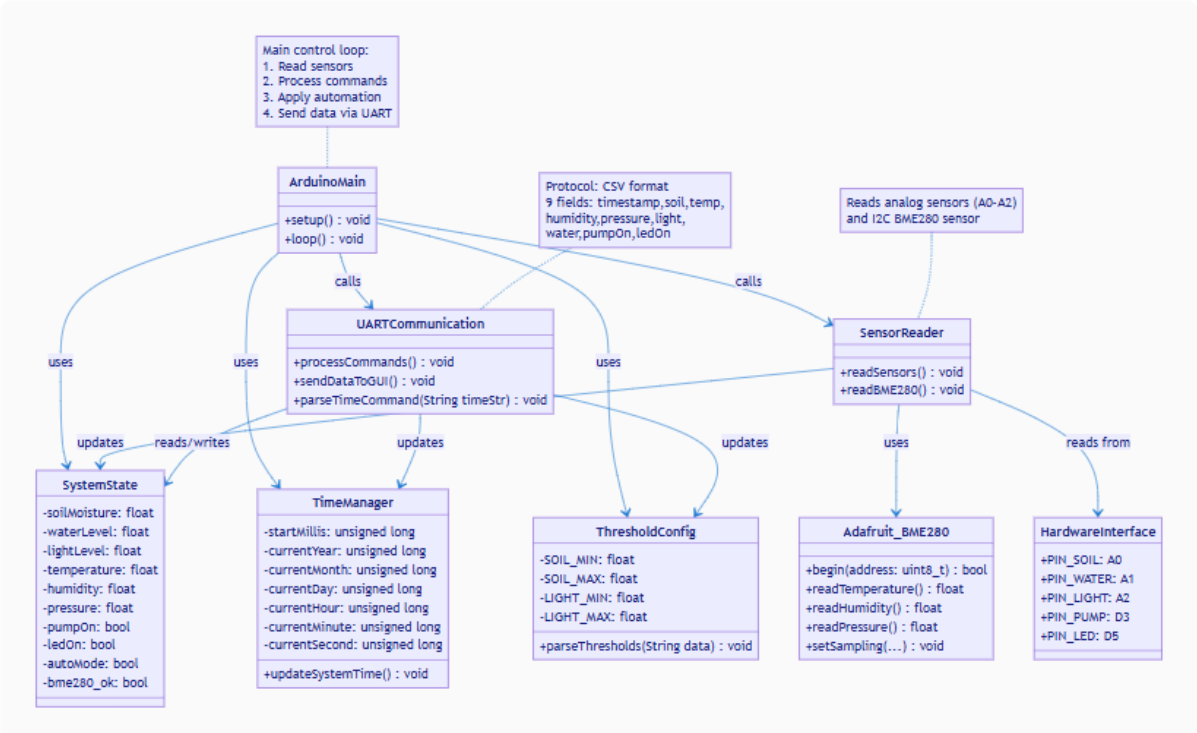
Przygotowano kompletną dokumentację montażu:

- SCHEMAT_POLACZEN.md – pełny schemat elektryczny (15 stron)
- QUICK_START.md – instrukcja montażu krok-po-kroku (5 stron)
- Specyfikacje pinów Arduino Nano, moduł PWM MOSFET, wszystkie czujniki

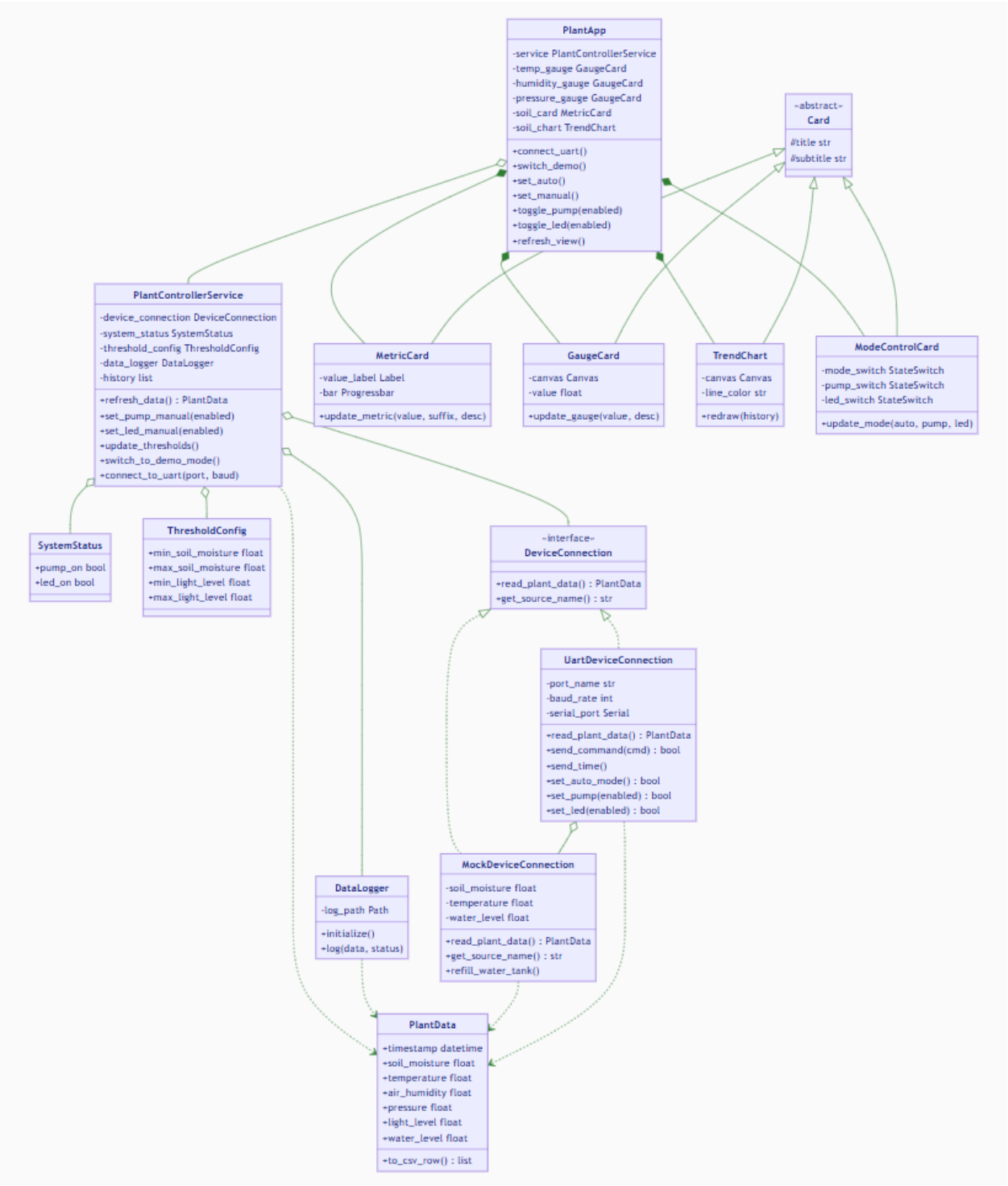
3. Postępy w Części Programowej (Student A)

3.1 Schemat UML programów

Arduino
https://khusiatynski.github.io/smart-plant-system/schemat/arduino_uml.html



GUI Python
https://khusiatynski.github.io/smart-plant-system/schemat/gui_python_uml.html


PlantApp
-service PlantControllerService
-temp_gauge GaugeCard
-humidity_gauge GaugeCard
-pressure_gauge GaugeCard
-soil_card MetricCard
-soil_chart TrendChart
+connect_uart()
+switch_demo()
+set_auto()
+set_manual()
+toggle_pump(enabled)
+toggle_led(enabled)
+refresh_view()
~abstract~
Card
#title str
#subtitle str
PlantControllerService
-device_connection DeviceConnection
-system_status SystemStatus
-threshold_config ThresholdConfig
-data_logger DataLogger
-history list
+refresh_data() : PlantData
+set_pump_manual(enabled)
+set_led_manual(enabled)
+update_thresholds()
+switch_to_demo_mode()
+connect_to_uart(port, baud)
MetricCard
-value_label Label
-bar Progressbar
+update_metric(value, suffix, desc)
GaugeCard
-canvas Canvas
-value float
+update_gauge(value, desc)
TrendChart
-canvas Canvas
-line_color str
+redraw(history)
ModeControlCard
-mode_switch StateSwitch
-pump_switch StateSwitch
-led_switch StateSwitch
+update_mode(auto, pump, led)
SystemStatus
+pump_on bool
+led_on bool
ThresholdConfig
+min_soil_moisture float
+max_soil_moisture float
+min_light_level float
+max_light_level float
~interface~
DeviceConnection
+read_plant_data() : PlantData
+get_source_name() : str
UartDeviceConnection
-port_name str
-baud_rate int
-serial_port Serial
+read_plant_data() : PlantData
+send_command(cmd) : bool
+send_time()
+set_auto_mode() : bool
+set_pump(enabled) : bool
+set_led(enabled) : bool
MockDeviceConnection
-soil_moisture float
-temperature float
-water_level float
+read_plant_data() : PlantData
+get_source_name() : str
+refill_water_tank()
DataLogger
-log_path Path
+initialize()
+log(data, status)
PlantData
+timestamp datetime
+soil_moisture float
+temperature float
+air_humidity float
+pressure float
+light_level float
+water_level float
+to_csv_row() : list

3.2. Kod Arduino

Zaimplementowano kompletny kod sterujący (sketch_gui_sync.ino) zawierający:

- Obsługę czujnika BME280 (I2C, adresy 0x76/0x77) z biblioteką Adafruit_BME280
- Odczyt czujników analogowych: gleba (A0), woda (A1), światło (A2) z mapowaniem 0-100%
- Sterowanie PWM: pompa (D3), LED grow (D5)
- Algorytm automatyki z histerezą dla wilgotności gleby i poziomu światła
- Zabezpieczenie pompy przed suchobiegiem (wyłączenie przy poziomie wody ≤2%)
- Komunikacja UART – protokół CSV 9 pól: timestamp, soil, temp, humidity, pressure, light, water, pumpOn, ledOn
- Synchronizacja czasu przez SET_TIME z GUI
- Komendy sterujące: AUTO, MANUAL, PUMP:ON/OFF, LED:ON/OFF, SET_THRESHOLDS

3.3. Aplikacja GUI Python

Opracowano pełnoprawną aplikację desktopową (GUI/app.py) z następującymi funkcjami:

- Panel główny – ciemny interfejs (tło #08111f), przewijalny widok, responsywny layout
- Zegary BME – 3 okrągłe gauges dla temperatury, wilgotności powietrza i ciśnienia (kolorowa wizualizacja)
- Metryki rośliny – karty z paskami postępu dla wilgotności gleby, światła, poziomu wody
- Wykresy trendów – historia 40 ostatnich pomiarów dla gleby i światła
- Sterowanie trybu – kolorowe przełączniki AUTO/RĘKA z wyraźnym stanem (zielony ON, czerwony OFF)
- Konfiguracja progów – edycja zakresów automatyki (min/max dla gleby i światła)
- Połączenie UART – wybór portu COM, baud rate, tryb demo z symulacją
- Logowanie CSV – automatyczny zapis wszystkich pomiarów do logs/plant-log.csv

3.4. Symulacja Wokwi

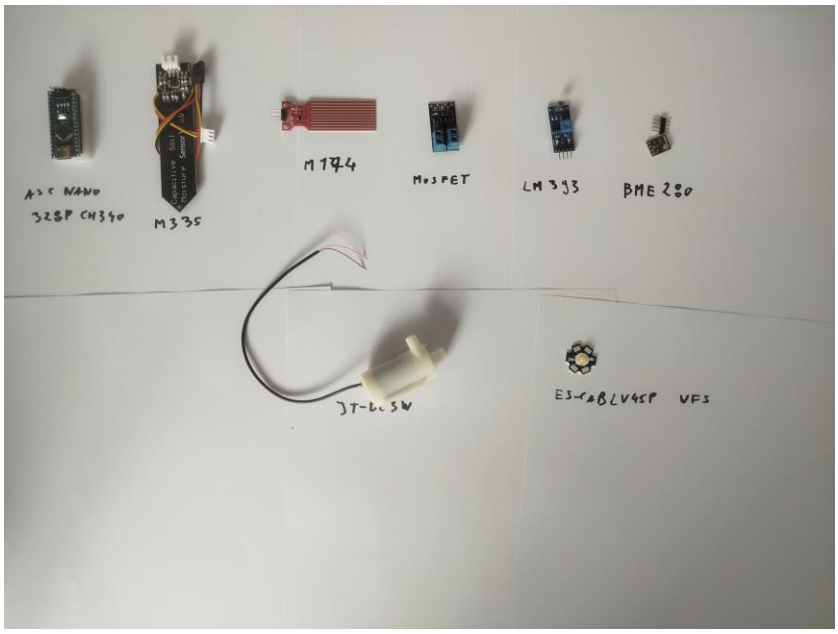
Utworzono działający projekt symulacyjny:

- URL: https://wokwi.com/projects/461119979508150273
- Własny chip BME280 (kod JSON) z poprawną kalibracją
- 2x slide potentiometer (gleba A0, woda A1) + fotoresystor (światło A2)
- LED czerwona (pompa D3), LED biała (LED grow D5)
- Etykiety tekstowe (wokwi-text) dla czytelności układu

4. Postępy w Części Sprzętowej (Student B)

4.1. Dobór i zakup komponentów

Student B dokonał analizy i zakupu elementów elektronicznych zgodnie z założeniami projektu. Kluczową decyzją było zastosowanie gotowego modułu PWM MOSFET zamiast lutowania pojedynczych tranzystorów, co znacząco uprościło montaż i zwiększyło niezawodność układu.



4.2. Opracowanie schematów połączeń

Przygotowano szczegółową dokumentację montażową:

- Schemat blokowy – ASCII art pokazujący strukturę systemu
- Pinout Arduino Nano – przypisanie wszystkich pinów z komentarzami
- Sekcja zasilania – krytyczne uwagi o osobnym zasilaczu 5V 2A dla pompy i LED
- Połączenia czujników – szczegółowe diagramy dla BME280, gleby, wody, światła
- Sekcja mocy – schemat modułu PWM MOSFET z wbudowanymi diodami flyback
- Obliczenia rezystora LED – wymiar 2.2Ω 5W dla LED 3W @ 700mA

4.3. Specyfikacja modułu PWM MOSFET

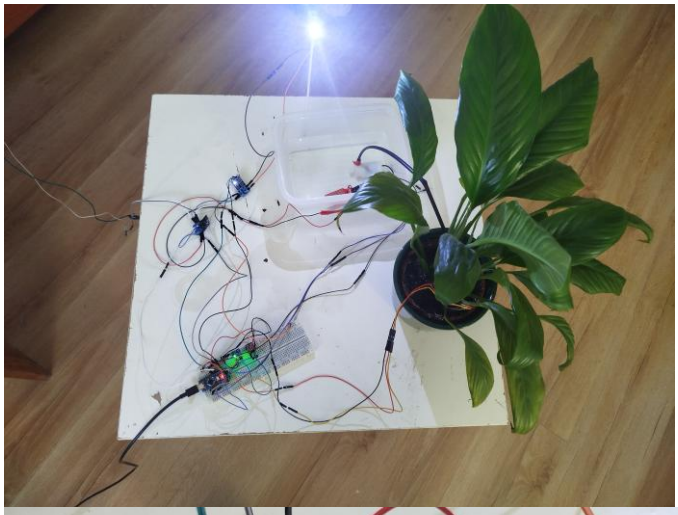
Zastosowany moduł XY-MOS (2x AOD4184A) charakteryzuje się:

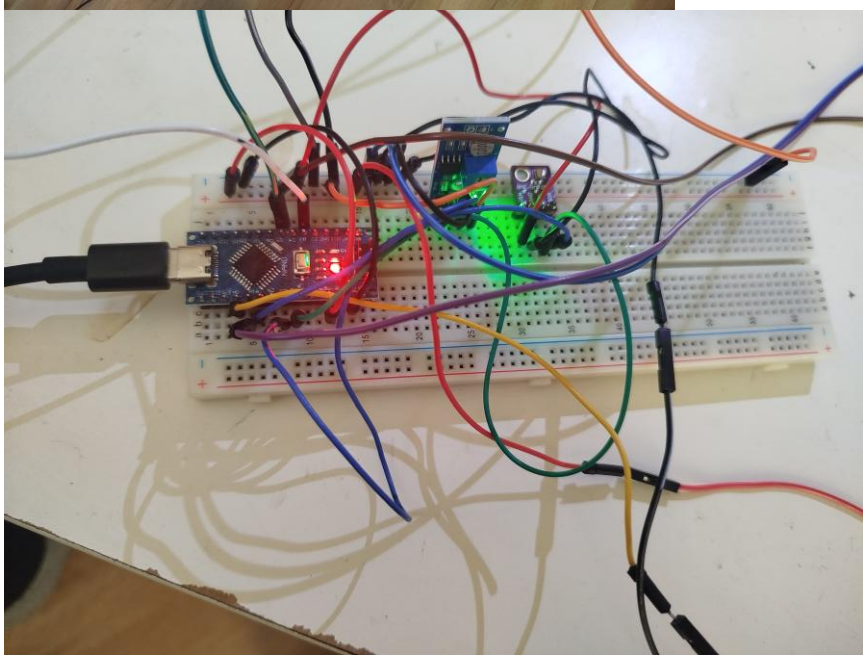
- Napięciem sterowania 3.3-20V (logic-level, kompatybilny z Arduino)
- Napięciem zasilania 5-36V DC
- Obciążeniem ciągłym 15A na kanał (pompa ~0.5A + LED ~0.7A = bezpieczny margines)
- Wbudowanymi diodami flyback (eliminacja zewnętrznych zabezpieczeń)
- Złączami śrubowymi (łatwy montaż bez lutowania)

4.4 Montaż układu

Przygotowano prototyp układu :

- Montaż układu według schematu QUICK_START.md
- Podłączenie wszystkich czujników do Arduino Nano
- Montaż pompy wodnej i oświetlenia LED
- Lutowanie połączeń
- Test LED grow – sterowanie PWM przez Arduino
- Test pompy – uruchomienie komendą PUMP:ON





5. Napotkane Problemy i Rozwiązania

Problem	Rozwiązanie
BME280 zwracał błędne odczyty (-39.3°C, 0.8%)	Przełączono na bibliotekę Adafruit_BME280 z poprawną kalibracją
Etykiety w Wokwi nie były widoczne podczas symulacji	Użyto komponentów wokwi-text zamiast attrs.label
Format danych Arduino nie pasował do GUI (brak pola pressure)	Zaktualizowano kod Arduino o wysyłanie ciśnienia (9 pól CSV)
Wyszukiwanie adresu BME280 (0x76 lub 0x77)	Kod automatycznie próbuje oba adresy podczas inicjalizacji
Brak fontów polskich w ReportLab	Użyto DejaVu Sans z pełnym wsparciem znaków Unicode
Brak odpowiedniego rezystora	Zastosowanie tymczasowo innego o wieksej rezystancji

6. Obecny Stan Projektu

6.1. Ukończone elementy

Element	Status	%
Kod Arduino	Ukończony	100%
Aplikacja GUI Python	Ukończona	100%
Symulacja Wokwi	Działająca	100%
Schemat elektryczny	Ukończony	100%
Dokumentacja montażu	Ukończona	100%
Zakup komponentów	Zrealizowany	100%
Wstępny montarz układu	Działający	80%
Logowanie danych CSV	Działające	100%
Komunikacja UART	Gotowa	100%

6.2. Do dokupienia

Aby ukończyć fizyczną realizację układu, niezbędne jest dokupienie:

- Rezystor mocy 2.2 Ω 5W – krytyczny element dla LED 3W (bez niego LED ulegnie przepaleniu)- układ tymczasowo podłączony z dostępnym rezystorem 15 Ω
- Zasilacz 5V 2A – osobne zasilanie dla pompy i LED
- Radiator aluminiowy dla LED 3W (opcjonalnie, ale zalecane)
- Pasta termoprzewodząca do radiatora
- Czulnik ultradźwiękowy

7. Planowane Dalsze Kroki

7.1. Montaż fizyczny (Student B)

Po dostarczeniu brakujących elementów:

- Lutowanie rezystora 2.2Ω 5W szeregowo z LED 3W
- Podłaczenie czytnika kart SD
- Wymiana czujnika poziomu wody na czujnik bezdotykowy

7.2. Testy integracyjne (wspólnie)

- Test czujników – weryfikacja odczytów w Serial Monitor
- Test trybu AUTO – automatyczne włączanie pompy i LED według progów
- Test komunikacji UART – synchronizacja z GUI Python
- Kalibracja czujników – dostrojenie zakresów analogowych

7.3. Finalizacja (Student A)

- Optymalizacja GUI – ewentualne poprawki na podstawie testów
- Dokumentacja użytkownika końcowego
- Przygotowanie prezentacji projektu
- Raport końcowy z wynikami testów

8. Podsumowanie

Projekt znajduje się w zaawansowanym stadium realizacji. Ukończono wszystkie główne komponenty programowe oraz przygotowano kompletną dokumentację techniczną. Część programowa (Student A) jest w 100% gotowa i przetestowana w środowisku symulacyjnym Wokwi. Część sprzętowa (Student B) została w pełni zaplanowana i udokumentowana, większość elementów zakupiono, dokonano wstępnego montażu.

Osiągnięcia projektu:

- Kompleksowa aplikacja GUI w Python z ciemnym interfejsem, zegarami BME i wykresami trendów
- Pełna implementacja kodu Arduino z obsługą wszystkich czujników i komunikacją UART
- Działająca symulacja Wokwi dostępna publicznie online
- Szczegółowe schematy elektryczne (15 stron) i Quick Start Guide (5 stron)
- Logowanie danych do CSV z pełną sygnaturą czasową
- Automatyka z histerezą i zabezpieczeniami (pompa, LED)

Do realizacji pozostaje:

- Dokupienie rezystora mocy 2.2Ω 5W, zasilacza 5V 2A i akcesoriów
- Testy integracyjne na rzeczywistym sprzęcie

Zespół przewiduje ukończenie projektu zgodnie z harmonogramem po dostarczeniu brakujących elementów pasywnych. Dotychczasowe prace przebiegały sprawnie dzięki jasnemu podziałowi ról i systematycznej współpracy między członkami zespołu.